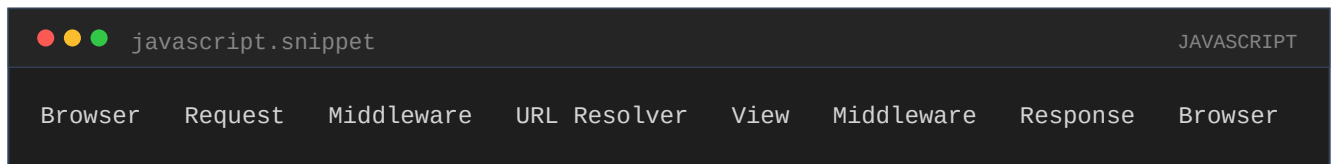


Django Middleware -- Complete Beginner to Advanced Notes

1. What is Django Middleware?

Middleware is a layer in Django's request/response processing system that can process a request before it reaches the view and process the response before it reaches the client.

Simply:



Middleware can be used for application-wide concerns such as:

```
javascript.snippet JAVASCRIPT

AuthenticationLoggingSecurityRequest trackingRate limitingCustom headersMaintenance
modeMulti
-tenancy
```

2. Where is Middleware Configured?

Middleware is configured in the project's:

```
javascript.snippet JAVASCRIPT

settings.py
```

file, inside the MIDDLEWARE list.

Example:

```
javascript.snippet JAVASCRIPT

MIDDLEWARE = [    "django.middleware.security.SecurityMiddleware",
    "django.contrib.sessions.middleware.SessionMiddleware"
    ,    "django.middleware.common.CommonMiddleware",
    "django.middleware.csrf.CsrfViewMiddleware"
    ,    "django.contrib.auth.middleware.AuthenticationMiddleware",
    "django.contrib.messages.middleware.MessageMiddleware"
    ,]

```

These are Django's built-in middleware classes.

3. Why is Middleware Used?

Middleware is generally used for functionality that needs to apply to **many or nearly all requests**.

Common use cases:

```
javascript.snippet JAVASCRIPT

Request LoggingAuthenticationSecurityCSRF ProtectionSession HandlingIP FilteringRate
LimitingRequest
TimingCustom HeadersMaintenance ModeMulti-Tenant DetectionRequest/Response
Modification

```

4. Important Built-in Middleware

4.1 SecurityMiddleware

```
javascript.snippet JAVASCRIPT

"django.middleware.security.SecurityMiddleware"

```

SecurityMiddleware helps handle various security-related behaviors.

Depending on the configuration, it can provide security-related HTTP behavior and HTTPS-related redirects.

5. SessionMiddleware

```
javascript.snippet JAVASCRIPT
"django.contrib.sessions.middleware.SessionMiddleware"
```

SessionMiddleware provides support for Django sessions.

Example:

```
javascript.snippet JAVASCRIPT
request.session["name"] = "Faruk"
```

Later:

```
javascript.snippet JAVASCRIPT
name = request.session.get("name")
```

can be used to retrieve the value.

6. CsrfViewMiddleware

```
javascript.snippet JAVASCRIPT
"django.middleware.csrf.CsrfViewMiddleware"
```

CsrfViewMiddleware provides CSRF protection.

In a Django form, you normally use:

```
javascript.snippet JAVASCRIPT
<form method="post">    {% csrf_token %}    ...</form>
```

The CSRF token helps protect POST requests against Cross-Site Request Forgery attacks.

7. AuthenticationMiddleware

```
javascript.snippet JAVASCRIPT
"django.contrib.auth.middleware.AuthenticationMiddleware"
```

AuthenticationMiddleware makes authenticated user information available through the request.

In a view:

```
javascript.snippet JAVASCRIPT
request.user
```

can be used.

Example:

```
javascript.snippet JAVASCRIPT
if request.user.is_authenticated:    print(request.user)
```

8. MessageMiddleware

```
javascript.snippet JAVASCRIPT
"django.contrib.messages.middleware.MessageMiddleware"
```

MessageMiddleware is used by Django's messages framework.

Example:

```
javascript.snippet JAVASCRIPT
from django.contrib import messages
messages.success(request,
    "Profile updated successfully."
)
```

9. Creating Custom Middleware

Suppose we want to print a message for every request.

Project structure:

```
javascript.snippet JAVASCRIPT
project/ manage.py myproject/ settings.py myapp/ middleware.py views.py
```

middleware.py:

```
javascript.snippet JAVASCRIPT
class MyMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response
    def __call__(self, request):
        print("Request received")
        response = self.get_response(request)
        print("Response sent")
        return response
```

10. Middleware `__init__()`

```
javascript.snippet JAVASCRIPT
def __init__(self, get_response):
    self.get_response = get_response
```

`__init__()` is called when the middleware instance is created.

It is generally used for:

```
javascript.snippet JAVASCRIPT
ConfigurationInitializationMiddleware setup
```

Important:

`__init__()` is generally not called for every request. It is used when the middleware instance is initialized.

11. Middleware `__call__()`

`__call__()` is the most important method in modern custom Django middleware.

```
javascript.snippet JAVASCRIPT

def __call__(self, request):    print("Before view
")    response = self.get_response(request)    print("
After view")    return response
```

Here:

```
javascript.snippet JAVASCRIPT

self.get_response(request)
```

passes the request to the next middleware and eventually to the view.

The response then travels back through the middleware layers.

12. Middleware Onion Model

Django middleware can be understood as an **onion-like layered structure**.

Suppose we have:

```
javascript.snippet JAVASCRIPT

Middleware
```

During the request:

```
javascript.snippet JAVASCRIPT

Request    Middleware 1    Middleware 2    Middleware 3    View
```

During the response:

```
javascript.snippet JAVASCRIPT

View    Middleware 3    Middleware 2    Middleware 1    Response
```

Therefore, outer middleware can execute code both **before and after the view**.

13. Middleware Execution Order

The order of middleware in:

```
javascript.snippet JAVASCRIPT

MIDDLEWARE = [ ... ]
```

is important.

Generally, during the request phase, middleware is entered from top to bottom.

During the response phase, the middleware layers are traversed in reverse order.

```
javascript.snippet JAVASCRIPT

Request  M1  M2  M3  View  M3  M2  M1  Response
```

14. Middleware Short-Circuit

Middleware can prevent a request from reaching the view by returning a response directly.

Example:

```
javascript.snippet JAVASCRIPT

from django.http import HttpResponse
class MaintenanceMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response
    def __call__(self, request):
        maintenance = True
        if maintenance:
            return HttpResponse("Site under maintenance")
        return self.get_response(request)
```

Flow:

```
javascript.snippet JAVASCRIPT

Request  Middleware  Maintenance?  Yes  Response  No  View
```

In this case, the view will not execute.

15. Middleware Hooks

There are two important ways to understand Django middleware:

Modern Middleware Style

```
javascript.snippet JAVASCRIPT

__init__().__call__()
```

Optional / Legacy-Style Hooks

```
javascript.snippet JAVASCRIPT

process_request()process_view()process_exception()process_template_response()
process_response
()
```

The process_* hooks are commonly used with `MiddlewareMixin`.

16. process_view()

`process_view()` is called before the view is executed.

It is commonly used through `MiddlewareMixin`.

Example:

```
javascript.snippet JAVASCRIPT

from django.utils.deprecation import MiddlewareMixinclass ViewMiddleware(
MiddlewareMixin
):
    def process_view(
        self,
        request,
        view_func,
        view_args
        ,
        view_kwargs,
    ):
        print(
            "View:",
            view_func.
            __name__
        ,
        )
```

Parameters:

Parameter	Meaning
request	Current request
view_func	The view function that will be called
view_args	Positional arguments
view_kwargs	Keyword arguments

17. When Should You Use `process_view()`?

Common examples:

 javascript.snippet

JAVASCRIPT

Permission checksRequest loggingView-specific checksCustom access control

However, for authentication functionality, Django's existing authentication system is generally more appropriate.

18. `process_exception()`

`process_exception()` can be used when an exception occurs during view or downstream processing.

Example:

 javascript.snippet

JAVASCRIPT

```
from django.utils.deprecation import MiddlewareMixin
class ExceptionMiddleware(MiddlewareMixin):
    def process_exception(self, request, exception):
        print("Exception:", exception)
```

Common uses:

 javascript.snippet

JAVASCRIPT

Error loggingMonitoringCustom exception handling

19. `process_template_response()`

`process_template_response()` can be used when a view returns a `TemplateResponse`.

Example:

```
javascript.snippet JAVASCRIPT

from django.utils.deprecation import MiddlewareMixin
class TemplateMiddleware(MiddlewareMixin):
    def process_template_response(self, request, response):
        response.context_data["site_name"] = ("My Website")
    return response
```

Important

This does **not** run for every type of response.

It is specifically useful for responses such as `TemplateResponse`.

20. `process_request()` and `process_response()`

When using `MiddlewareMixin`, you can use the older-style hooks.

Example:

```
javascript.snippet JAVASCRIPT

from django.utils.deprecation import MiddlewareMixin
class MyMiddleware(MiddlewareMixin):
    def process_request(self, request):
        print("Before view")
    def process_response(self, request, response):
        print("After view")
    return response
```

For new custom middleware, understanding and using the modern `__call__()` style is generally more important.

21. Middleware Hooks Summary

Method	Purpose
<code>__init__()</code>	Middleware initialization
<code>__call__()</code>	Main request/response wrapper
<code>process_request()</code>	Request processing before the view
<code>process_view()</code>	Processing before view execution
<code>process_exception()</code>	Exception handling
<code>process_template_response()</code>	TemplateResponse processing
<code>process_response()</code>	Response processing

The behavior of `process_*` methods depends on the middleware implementation and execution order. `__call__()` is the core mechanism of modern middleware.

22. Request Logging Middleware

A simple logging middleware:

```

javascript.snippet
JAVASCRIPT

import logging
logger = logging.getLogger(__name__)

class LoggingMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        logger.info("Request: %s %s", request.method, request.path)
        response = self.get_response(request)
        logger.info("Status: %s", response.status_code)
        return response

```

Instead of `print()`, Python's logging module is more appropriate for production applications.

23. Request Time Measurement

To measure how long a request takes to process:

```
javascript.snippet JAVASCRIPT

import time
class TimerMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response
    def __call__(self, request):
        start = time.perf_counter()
        response = self.get_response(request)
        duration = (time.perf_counter() - start)
        print(f"{request.path}: {duration:.4f}s")
        return response
```

Uses:

```
javascript.snippet JAVASCRIPT

Performance monitoringSlow endpoint detectionDebugging
```

24. Custom Response Header

A middleware can add a custom header to the response.

```
javascript.snippet JAVASCRIPT

class CustomHeaderMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response
    def __call__(self, request):
        response = self.get_response(request)
        response["X-App-Name"] = "My Django App"
        return response
```

Response:

```
javascript.snippet JAVASCRIPT

X-App-Name: My Django App
```

25. IP Blocking Middleware

Simple example:

```
javascript.snippet JAVASCRIPT

from django.http import HttpResponse
class BlockIPMiddleware:
    BLOCKED_IPS = {
        "192.168.1.10"
    }
    def __init__(self, get_response):
        self.get_response = get_response
    def __call__(self, request):
        ip = request.META.get("REMOTE_ADDR")
        if ip in self.BLOCKED_IPS:
            return HttpResponse("Access Denied", status=403)
        return self.get_response(request)
```

Important

In a production environment, REMOTE_ADDR may not always contain the actual client IP because of proxies or load balancers.

Therefore, forwarded IP headers should **not be blindly trusted**.

26. Authentication / Login Check Middleware

A simple example:

```
javascript.snippet JAVASCRIPT

from django.shortcuts import redirect
class LoginRequiredMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response
    def __call__(self, request):
        public_paths = {
            "/login/",
            "/register/",
        }
        if (
            not request.user.is_authenticated
            and request.path not in public_paths
        ):
            return redirect("login")
        return self.get_response(request)
```

Important

In real projects, Django's built-in authentication tools and mechanisms such as:

```
javascript.snippet JAVASCRIPT

login_requiredLoginRequiredMixin
```

are often more appropriate.

If global middleware is used for login protection, exceptions for admin, login, static/media,

API, and public pages must be handled carefully.

27. API Authentication Middleware

Middleware can technically be used to check a token or API key.

Example:

```
javascript.snippet JAVASCRIPT

from django.http import JsonResponse
class APIKeyMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response
    def __call__(self, request):
        api_key = request.headers.get('X-API-Key')
        if request.path.startswith('/api/'):
            if api_key != 'expected-key':
                return JsonResponse(
                    {'error': 'Invalid API key'},
                    status=401,
                )
        return self.get_response(request)
```

Important

In a real application, an API key should not be hard-coded.

Use appropriate mechanisms such as:

```
javascript.snippet JAVASCRIPT

Environment variables
Secret management
Database-backed credentials
```

28. JWT Authentication Middleware

JWT authentication middleware can be implemented manually.

However, in a Django REST Framework project, using an established authentication package is generally more appropriate.

Conceptual flow:

```
javascript.snippet JAVASCRIPT

Request Authorization Header Extract Bearer Token Verify Signature Decode
Claims
Find User Attach Authentication Information View
```

Important

Simply doing:

```
javascript.snippet JAVASCRIPT

jwt.decode(...)
```

does not create a complete authentication system.

You also need to consider:

```
javascript.snippet JAVASCRIPT

Signature verificationExpirationIssuerAudienceToken typeError handlingUser
lookupAuthorizationToken
revocation strategy
```

29. Rate Limiting Middleware

The purpose of rate limiting is to restrict the frequency of requests from a particular client, user, or IP.

Concept:

```
javascript.snippet JAVASCRIPT

Client Request Rate Limiter Within limit View Exceeded 429
```

Example response:

```
javascript.snippet JAVASCRIPT

from django.http import JsonResponse
return JsonResponse( { "error":
    "Too many requests"
}, status=429,)
```

Important

An implementation such as:

```
javascript.snippet JAVASCRIPT
requests = {}
```

or:

```
javascript.snippet JAVASCRIPT
last_request = {}
```

is not a production-ready rate limiter.

Potential problems include:

```
javascript.snippet JAVASCRIPT
Multiple workersMultiple serversProcess restartMemory usageRace
conditionsExpirationDistributed
state
```

For production systems, Redis-based rate limiting or an established framework/library is generally more appropriate.

30. Maintenance Mode Middleware

Simple example:

```
javascript.snippet JAVASCRIPT
from django.http import HttpResponse
class MaintenanceMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response
    def __call__(self, request):
        maintenance = True
        if maintenance:
            return HttpResponse(
                "Site under maintenance",
                status=503,
            )
        return self.get_response(request)
```

HTTP 503 Service Unavailable is more appropriate for maintenance mode.

31. Multi-Tenant Middleware

In a SaaS application, a tenant can be identified from a subdomain.

Example:

javascript.snippetJAVASCRIPT

```
company1.example.comcompany2.example.com
```

Middleware:

javascript.snippetJAVASCRIPT

```
class TenantMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        host = request.get_host()
        request.tenant_host = host
        return self.get_response(request)
```

In a real multi-tenant application, simply assigning a string does not create tenant isolation.

You also need to consider:

javascript.snippetJAVASCRIPT

```
Tenant lookupDatabase isolationAuthorizationData filteringCachingSubdomain validationSecurity
```

32. Middleware vs Decorator vs Signals

Feature	Middleware	Decorator	Signal
Scope	Global/request-level	Specific view/function	Event-based
Trigger	Request/response lifecycle	Function/view call	Specific event
Common use	Logging, security	View-specific behavior	Model/application events
Example	Request logging	login_required	post_save

33. Middleware Example

For all requests:

```
javascript.snippet JAVASCRIPT

class LogMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response
    def __call__(self, request):
        print(request.path)
        return self.get_response(request)
```

34. Decorator Example

For a specific view:

```
javascript.snippet JAVASCRIPT

from functools import wraps
def my_decorator(view_func):
    @wraps(view_func)
    def wrapper(request, *args, **kwargs):
        print("Before view")
        return view_func(request, *args, **kwargs)
    return wrapper
```

Usage:

```
javascript.snippet JAVASCRIPT

@my_decorator
def home(request):
    ...
```

35. Signal Example

Signals can be used for model or application events.

```
javascript.snippet JAVASCRIPT

from django.db.models.signals import post_save
from django.dispatch import receiver
from .models import Product
@receiver(post_save, sender=Product)
def product_created(sender, instance, created, **kwargs):
    if created:
        print("New product:", instance.name, )
```

When a Product is saved, the signal handler can execute.

36. Middleware vs Decorator vs Signal -- Easy Rule

Remember:

```
javascript.snippet
```

JAVASCRIPT

```
Middleware Request/Response level
Decorator Specific function/view level
Signal Event level
```

Examples:

```
javascript.snippet
```

JAVASCRIPT

```
Want to log every request Middleware
Want to protect only the checkout view
Decorator
Want to do something when a Product is created Signal
```

37. Async Middleware

Modern Django supports asynchronous middleware.

Example:

```
javascript.snippet
```

JAVASCRIPT

```
class AsyncMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response
    async def __call__(self, request):
        print("Before view")
        response = await self.get_response(request)
        print("After view")
        return response
```

This can work with asynchronous request handling.

Important

Using async middleware does **not automatically make an application faster**.

Performance benefits depend on factors such as:

```
javascript.snippet JAVASCRIPT
I/O workloadAsync viewsDatabase behaviorExternal API callsMiddleware designServer
configuration
```

38. Middleware and WebSocket

When using Django Channels, WebSocket requests are handled through the ASGI/Channels stack.

Traditional Django middleware and Channels WebSocket middleware are not exactly the same thing.

For WebSocket authentication, Channels middleware such as:

```
javascript.snippet JAVASCRIPT
AuthMiddlewareStack
```

can be used.

Architecture:

```
javascript.snippet JAVASCRIPT
Browser  WebSocket  ASGI  Channels Middleware  Consumer
```

39. Production-Level Logging Middleware

In production environments, use logging instead of print().

```

javascript.snippet
JAVASCRIPT

import logging
import time
logger = logging.getLogger(__name__)

class LoggingMiddleware:

    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        start = time.perf_counter()
        response = self.get_response(request)
        duration = (time.perf_counter() - start)
        logger.info(
            "%s %s %s %.3fs", request.method, request.path,
            response.status_code, duration,
        )
        return response

```

Example log:

```

javascript.snippet
JAVASCRIPT

GET /products/ 200 0. /checkout/ 201 0.

```

40. Request Tracking with Middleware

Basic example:

```

javascript.snippet
JAVASCRIPT

from django.utils import timezone
class RequestTrackingMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response
    def __call__(self, request):
        print(
            request.method, request.path, timezone.now(),
        )
        response = self.get_response(request)
        return response

```

In production systems, be careful about logging sensitive information.

In particular, you should not log:

```

javascript.snippet
JAVASCRIPT

Passwords
Access tokens
Refresh tokens
OTP codes
Session secrets
Credit card information

```

41. Custom Security Headers

Security-related headers can be added to responses.

Example:

```
class SecurityHeadersMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response
    def __call__(self, request):
        response = self.get_response(request)
        response["X-Frame-Options"] = "DENY"
        response["X-Content-Type-Options"] = "nosniff"
        return response
```

However, before manually adding security headers, understand Django's built-in security settings and existing middleware configuration.

Important Correction

The old:

```
X-XSS-Protection
```

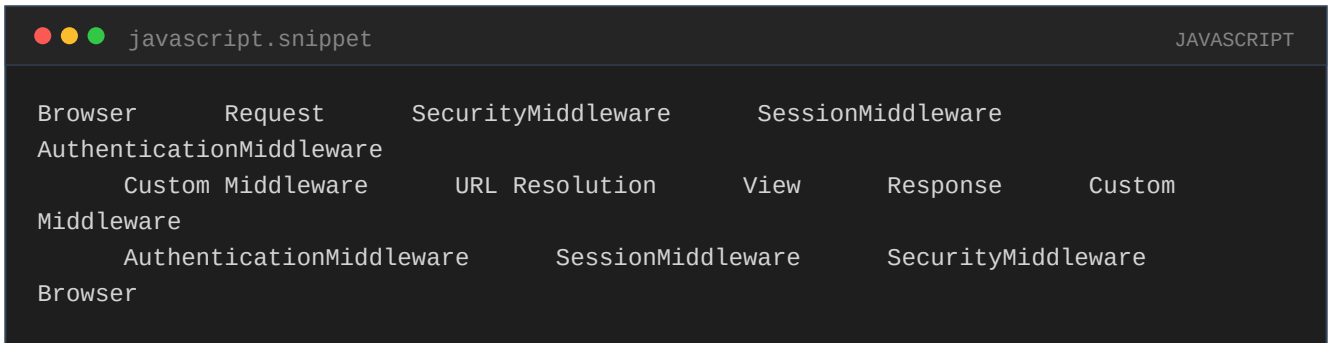
header should not be considered a primary modern browser security mechanism.

Modern applications should place more emphasis on:

```
Content Security Policy (CSP)
Proper output escaping
Secure cookie settings
HTTPS
Other appropriate security controls
```

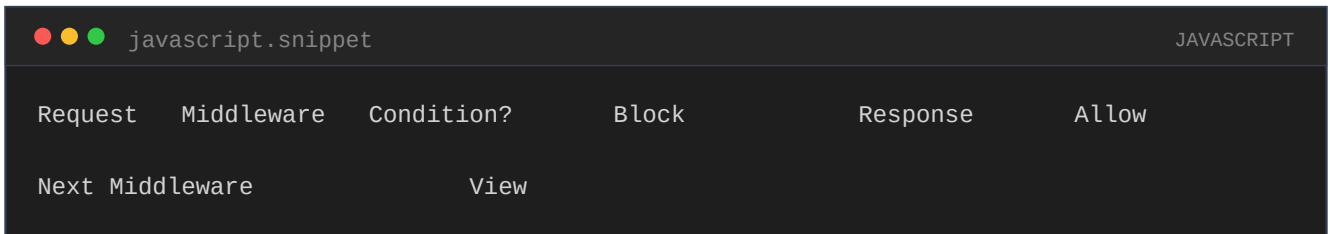
42. Complete Middleware Flow

A simplified flow:

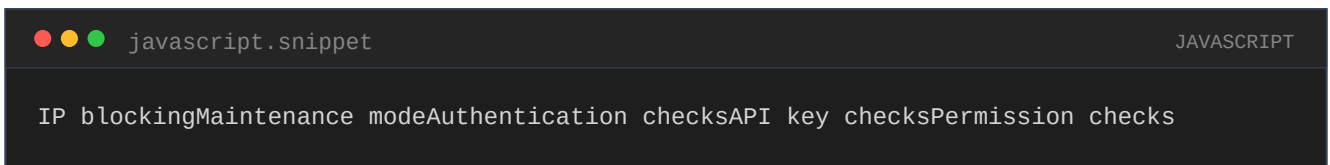


Actual execution details can vary depending on middleware, response type, exceptions, and Django configuration.

43. Middleware Short-Circuit Flow

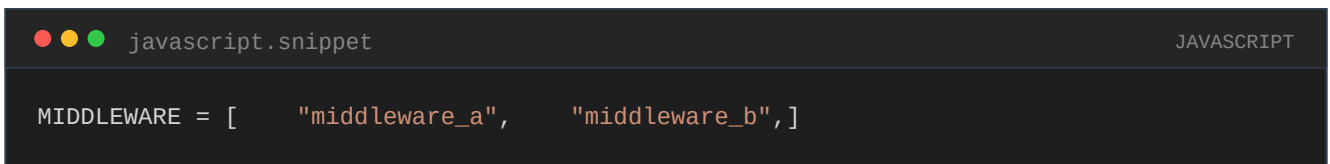


This can be used for:



44. Important Middleware Concepts

1. Order Matters



middleware_a acts as the outer layer and middleware_b acts as the inner layer.

2. Not Everything Should Be Done in Middleware

Business logic generally should not be placed inside middleware.

Bad examples:

```
Order calculationPayment processingComplex business rules
```

These are generally better placed in an appropriate service, view, or domain layer.

Middleware should primarily handle **cross-cutting concerns**.

45. Good Use Cases for Middleware

```
LoggingSecurityAuthentication infrastructureRequest tracingRequest timingHeadersTenant
detectionRate
limitingMaintenance mode
```

46. Good Middleware Design Principles

Middleware should ideally be:

```
SmallFocusedReusablePredictableFast
```

A single middleware should not contain 500 lines of business logic.

47. settings.py Example

```
MIDDLEWARE = [    "django.middleware.security.SecurityMiddleware",
"django.contrib.sessions.middleware.SessionMiddleware"
,    "django.middleware.common.CommonMiddleware",
"django.middleware.csrf.CsrfViewMiddleware"
,    "django.contrib.auth.middleware.AuthenticationMiddleware",
"django.contrib.messages.middleware.MessageMiddleware"
,    "myapp.middleware.LoggingMiddleware",]
```

The position of custom middleware should be carefully chosen based on what it depends on and what should run before or after it.

48. Middleware Project Structure

```
javascript.snippet JAVASCRIPT

project/ manage.py project/ settings.py urls.py asgi.py wsgi.py myapp/
__init__
.py middleware.py models.py views.py ...
```

middleware.py:

```
javascript.snippet JAVASCRIPT

class LoggingMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response
    def __call__(self, request):
        print(request.path)
        return self.get_response(request)
```

settings.py:

```
javascript.snippet JAVASCRIPT

MIDDLEWARE = [
    # Django middleware...
    "myapp.middleware.LoggingMiddleware",]
```

49. Interview Questions

Q1. What is Django Middleware?

Answer:

Middleware is a layer in Django's request/response processing system that can process a request before it reaches the view and process the response before it reaches the client.

Q2. Where is Middleware configured?

In:

```
javascript.snippet JAVASCRIPT
settings.py
```

inside the:

```
javascript.snippet JAVASCRIPT
MIDDLEWARE
```

list.

Q3. What are the main methods of Middleware?

The core methods of modern middleware are:

```
javascript.snippet JAVASCRIPT
__init__() __call__()
```

Q4. When is `__init__()` executed?

It is executed when the middleware instance is initialized.

Q5. When is `__call__()` executed?

It is executed during request processing, generally once for each request handled by that middleware.

Q6. What is `process_view()`?

It is a middleware hook that runs before the view is executed.

Q7. What is `process_exception()`?

It is a middleware hook that can be used for handling or logging exceptions raised during downstream processing.

Q8. What is `process_template_response()`?

It can be used to process a `TemplateResponse` returned by a view.

Q9. Can Middleware block a request?

Yes.

If middleware returns a response directly, the request does not continue to the view.

Q10. Middleware vs Decorator?

```
javascript.snippet JAVASCRIPT
Middleware Global/request-levelDecorator Specific view/function
```

Q11. Middleware vs Signal?

```
javascript.snippet JAVASCRIPT
Middleware Request/response lifecycleSignal Event-driven behavior
```

Q12. What is the Middleware execution order?

Request:

```
javascript.snippet JAVASCRIPT
Top Bottom
```

Response:

```
javascript.snippet JAVASCRIPT
Bottom Top
```

50. Quick Revision

Middleware

● ● ● javascript.snippet

JAVASCRIPT

Request Middleware View Middleware Response

Main Methods

● ● ● javascript.snippet

JAVASCRIPT

`__init__().__call__()`

Additional / Optional Hooks

● ● ● javascript.snippet

JAVASCRIPT

`process_request()process_view()process_exception()process_template_response()
process_response
()`

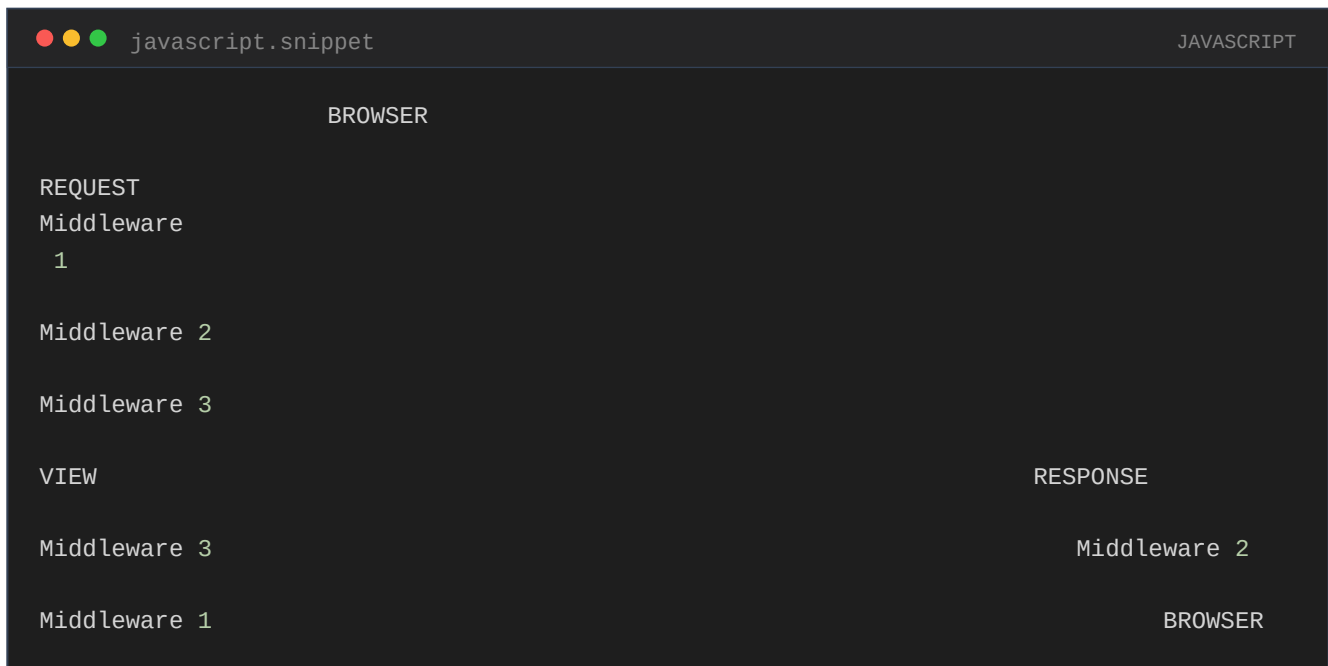
The process_* hooks are largely associated with legacy-style middleware support and are commonly used through MiddlewareMixin.

51. Real-World Use Cases

Use Case	Middleware's Role
Logging	Log requests/responses
Security	Security checks/headers
Authentication	Authentication infrastructure
Rate Limiting	Control request frequency
Monitoring	Request timing/tracing
Maintenance	Temporarily block requests
Multi-tenancy	Tenant identification

API Protection	API key/token checks
Custom Headers	Add response headers
Request Tracking	Track request metadata

52. Final Mental Model



One-Line Definitions

●●● javascript.snippet JAVASCRIPT

Middleware A processing layer in Django's request/response lifecycle. `__init__()` Initializes the middleware. `__call__()` The main request/response processing wrapper. `process_view()` A hook that runs before the view executes. `process_exception()` Used for exception handling or logging. `process_template_response()` Used for processing `TemplateResponse` objects.

MiddlewareMixin Provides support for legacy-style `process_*` middleware hooks. Decorator Modifies or wraps the behavior of a specific view/function. Signal Executes a handler when a specific event occurs. ASGI An interface for asynchronous-capable Python web applications and protocols. Channels A Django project that extends Django to handle WebSockets and other asynchronous communication use cases. Redis A common shared backend used for Channels channel layers.

Final Rule to Remember

●●● javascript.snippet JAVASCRIPT

Global Request/Response Concern	MiddlewareSpecific View Concern
DecoratorSpecific	
Application Event	Signal